

HANDWRITTEN DIGIT IDENTIFICATION USING MACHINE LEARNING APPROACH

Report submitted to

*Techno India University, West Bengal for
the partial fulfilment
of*

**Bachelor of Technology (B. Tech.) degree
in
Computer Science & Engineering**

By

Shahnihal Rahman

191001001260



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
TECHNO INDIA UNIVERSITY, WEST BENGAL,
SALT LAKE, KOLKATA – 700091, INDIA**

May 2023

ACKNOWLEDGEMENT

I would first like to thank my thesis supervisor Mr. Debashis Das (HOD), CSE Dept, TIU. He helped me whenever I ran into a trouble spot or had a question about my research or writing. I would also like to thank my peer Hriyam Dutta for his support.

I take this opportunity to express gratitude to all of the Department faculty members for their help and support.

I also thank my parents for the unceasing encouragement, support and attention and also sense of gratitude to one and all, who directly or indirectly, have bestowed their hand in this thesis.

Shahnihal Rahman

CERTIFICATE

This is to certify that the Dissertation Report entitled, "**HANDWRITTEN DIGIT IDENTIFICATION USING MACHINE LEARNING APPROACH**" submitted by "**SHAHNIHAL RAHMAN**" to Techno India University, Kolkata, India, is a record of Bonafede Project work carried out by them under my supervision and guidance and is worthy of consideration for the award of the degree of Bachelor of Technology (B.Tech) in Computer science & Engineering.

Approved By:

Supervisor(s)

Date:

HOD, CSE, Techno India University

Date:

Contents

1. Introduction	6
1.1 Objective.....	7
1.2 Problem specifications.....	8
1.3 Methodology.....	9
2. Literature Survey	11
3. Project Planning	14
4. Project Description	16
4.1 ML Model.....	16
4.2 Functional specification.....	17
4.2.1 Libraries used.....	17
4.2.2 Functions Performed.....	18
4.2.3 Limitations and restrictions.....	19
5. Working diagram and flowchart	22
6. Experimental Results & accuracy	24
7. Conclusion	28
8. References	29

Abstract

Handwritten digit recognition is a fundamental problem in the field of pattern recognition and machine learning. It involves the classification of images of handwritten digits into their respective numeric values. The k-nearest neighbors (KNN) algorithm is a popular and simple method for solving classification problems. This project explores the application of the KNN algorithm to handwritten digit recognition.

The project begins by acquiring a dataset of handwritten digit images, such as the MNIST dataset, which contains a large number of labeled examples. Each image is preprocessed to extract relevant features, such as pixel intensity values. The dataset is then divided into a training set and a test set. The training set is used to build the KNN model, while the test set is used to evaluate its performance.

The KNN algorithm works by comparing the features of a test image to those of the training images. It identifies the k nearest neighbors based on some distance metric, such as Euclidean distance. The majority class among the k neighbors is assigned as the predicted class for the test image. Various values of k are explored to find the optimal value that maximizes the accuracy of the classifier.

The project implements the KNN algorithm using a programming language such as Python. The scikit-learn library provides convenient tools for building and evaluating the KNN model. The accuracy of the model is evaluated using metrics such as classification accuracy, precision, recall, and F1 score.

The results of the project demonstrate the efficacy of the KNN algorithm for handwritten digit recognition. By tuning the value of k and optimizing the feature extraction process, high accuracy rates can be achieved. The project highlights the simplicity and interpretability of the KNN algorithm, making it a suitable choice for real-world applications where interpretability is crucial.

In conclusion, this project showcases the use of the KNN algorithm for handwritten digit recognition. It provides insights into the preprocessing steps, parameter selection, and evaluation metrics necessary for developing an accurate and efficient digit recognition system. The project's findings contribute to the broader field of pattern recognition and can be extended to other classification problems beyond handwritten digit recognition.

KEYWORDS: k-nearest neighbors (KNN) algorithm, classification, pattern recognition, machine learning, MNIST dataset

Introduction

Handwritten digit recognition is a significant problem in the field of pattern recognition and machine learning. It involves the task of correctly identifying and classifying handwritten digits into their respective numerical values. This problem has gained attention due to its wide range of applications, including optical character recognition (OCR), postal automation, digit-based data analysis, and digit recognition in forms.

The goal of handwritten digit recognition is to develop a system that can accurately interpret and classify handwritten digits, mimicking the human ability to recognize and distinguish different numerical symbols. Over the years, numerous machine learning techniques have been employed to tackle this problem, with varying degrees of success.

One such technique is the k-nearest neighbors (KNN) algorithm, which is a simple yet powerful approach for classification tasks. The KNN algorithm determines the class membership of a given data point by considering the class labels of its k nearest neighbors in the feature space. It relies on the assumption that similar data points tend to share the same class.

In this project, we aim to implement a handwritten digit recognition system using the KNN algorithm. The system will be designed to effectively classify handwritten digits by extracting suitable features from the digit images and utilizing the concept of proximity-based classification. The KNN algorithm offers advantages such as simplicity, interpretability, and adaptability to dynamic or online learning scenarios.

Throughout this project, we will explore various aspects such as data preprocessing, feature extraction, model training, parameter tuning, and performance evaluation. We will utilize libraries such as scikit-learn, NumPy, Matplotlib, and OpenCV to facilitate the implementation and analysis of the system.

By accurately recognizing and classifying handwritten digits, our system can contribute to various real-world applications, including digit-based data analysis, automated document processing, and improving the efficiency of postal services. It has the potential to enhance the accuracy and speed of digit recognition tasks, bringing us closer to achieving reliable and efficient solutions for handwritten digit recognition.

Objective

1. Acquiring a dataset of handwritten digit images, such as the MNIST dataset.
2. Preprocessing the images to extract relevant features.
3. Dividing the dataset into a training set and a test set.
4. Implementing the KNN algorithm using a programming language like Python.
5. Tuning the value of k and optimizing the feature extraction process.
6. Evaluating the accuracy of the KNN model
7. Providing insights and recommendations for developing an accurate and efficient handwritten digit recognition system.

Problem specification

The problem addressed in this project is handwritten digit recognition. Given a dataset of handwritten digit images, the goal is to develop a machine learning model using the k-nearest neighbors (KNN) algorithm that can accurately classify and recognize the digits in the images.

Specifically, the project aims to tackle the following challenges:

1. **Dataset Acquisition:** The project requires obtaining a suitable dataset of handwritten digit images. The MNIST dataset is commonly used for this purpose, but alternative datasets may also be considered.
2. **Preprocessing:** The raw images need to be preprocessed to extract relevant features. This may involve techniques such as resizing, normalization, and converting the images into a suitable format for analysis.
3. **Training and Test Set Split:** The dataset needs to be divided into a training set and a separate test set. The training set is used to train the KNN model, while the test set is used to evaluate its performance and generalization capabilities.
4. **KNN Algorithm Implementation:** The KNN algorithm needs to be implemented using a programming language such as Python. The distance metric, such as Euclidean distance, needs to be defined for calculating the similarity between the test image and the training images.
5. **Parameter Tuning:** The value of k , which determines the number of nearest neighbors considered for classification, needs to be tuned to find the optimal value that maximizes the accuracy of the KNN model. Other parameters or preprocessing steps may also need to be adjusted to improve performance.
6. **Evaluation Metrics:** The accuracy of the KNN model needs to be evaluated using appropriate metrics such as classification accuracy, precision, recall, and F1 score. These metrics provide insights into the model's performance in correctly classifying the digits.
7. **Comparison with Other Algorithms:** The performance of the KNN model needs to be compared with other popular classification algorithms to assess its effectiveness and identify potential strengths or weaknesses.

8. Real-World Applicability: The project aims to assess the interpretability and suitability of the KNN algorithm for real-world applications. Considerations such as computational efficiency, scalability, and interpretability are important factors to be addressed.

By addressing these challenges, the project seeks to develop an effective and accurate handwritten digit recognition system using the KNN algorithm. The results and insights gained from the project contribute to the broader field of pattern recognition and machine learning.

Methodology

1. Dataset Acquisition: Obtain a suitable dataset of handwritten digit images, such as the MNIST dataset, which consists of a large number of labeled examples. Alternatively, consider other relevant datasets that are publicly available or suitable for the task.

2. Preprocessing: Perform preprocessing steps on the images to extract relevant features. This may include resizing the images to a consistent size, normalizing the pixel values to a common range, and converting the images into a suitable format for analysis, such as grayscale.

3. Train-Test Split: Divide the dataset into a training set and a test set. Typically, a significant portion of the dataset, such as 70-80%, is used for training the model, while the remaining portion is used for testing and evaluating the model's performance.

4. Feature Extraction: Extract suitable features from the preprocessed images. For handwritten digit recognition, common features include pixel intensity values or more advanced techniques such as HOG (Histogram of Oriented Gradients) features.

5. KNN Algorithm Implementation: Implement the KNN algorithm using a programming language like Python. Utilize libraries such as scikit-learn, which provides efficient implementations of KNN and other machine learning algorithms. Define the distance metric, such as Euclidean distance, to measure the similarity between the feature vectors of training and test samples.

6. Model Training: Train the KNN model using the training set. During training, the model stores the feature vectors of the training samples along with their corresponding labels.

7. Model Evaluation: Evaluate the performance of the trained KNN model using the test set. Compute metrics such as classification accuracy, precision, recall, and F1 score to assess the model's ability to correctly classify the digits in the test images.

8. Parameter Tuning: Experiment with different values of k , which determines the number of nearest neighbors considered for classification. Perform cross-validation or use validation sets to select the optimal value of k that maximizes the model's performance.

9. Comparison with Other Algorithms: Compare the performance of the KNN model with other popular classification algorithms, such as Support Vector Machines (SVM), Random Forests, or Neural Networks. Evaluate their accuracy and other relevant metrics to identify the strengths and weaknesses of the KNN approach.

10. Real-World Applicability: Consider the computational efficiency, scalability, and interpretability of the KNN algorithm for real-world applications. Assess the trade-offs between accuracy and computational cost to determine if KNN is suitable for deployment in practical scenarios.

11. Iterative Refinement: Based on the evaluation results, fine-tune the preprocessing steps, feature extraction techniques, and parameters of the KNN model to improve its performance. Iterate and repeat the training and evaluation steps as necessary.

By following this methodology, the project aims to develop an effective handwritten digit recognition system using the KNN algorithm and provide insights into its performance and potential for real-world applications.

Literature survey

- Article: "A review on handwritten digit recognition techniques" by S. Kavitha and S. Radha. This article provides an overview of various techniques used for handwritten digit recognition, including KNN, neural networks, and support vector machines. It discusses the advantages and limitations of each approach and presents a comparative analysis of their performance.
- Book: "Pattern Recognition and Machine Learning" by Christopher M. Bishop. This comprehensive book covers various topics in pattern recognition, including handwritten digit recognition. It provides a detailed explanation of the KNN algorithm and its application in classification problems. It also discusses feature extraction, model evaluation, and comparison with other algorithms.
- Paper: "The MNIST Database of Handwritten Digits" by Y. LeCun et al. This seminal paper introduces the MNIST dataset, which has become a benchmark dataset for handwritten digit recognition. It describes the dataset, its preprocessing steps, and provides baseline results using different classification algorithms, including KNN.
- Article: "A Comparative Study of K-Nearest Neighbor, Support Vector Machine and Convolutional Neural Network for Handwritten Digit Recognition" by M. Hussain et al. This study compares the performance of KNN, support vector machines (SVM), and convolutional neural networks (CNN) for handwritten digit recognition. It evaluates their accuracy, training time, and memory usage, providing insights into the strengths and weaknesses of each approach.
- Paper: "Enhanced k-nearest neighbors' algorithm for handwritten digit recognition" by S. Mondal et al. This research paper presents an enhanced KNN algorithm by incorporating feature extraction techniques such as Zernike moments and Hu moments. The proposed method achieves improved accuracy in handwritten digit recognition compared to the traditional KNN approach.
- Article: "Feature Extraction for Handwritten Digit Recognition: A Review" by R. Ramesh and V. Kannan. This review article focuses on feature extraction techniques for handwritten digit recognition. It discusses popular methods such as pixel intensity values, shape-based features, and transform-based features. It provides a comprehensive comparison of these techniques in terms of accuracy and computational complexity.

- Paper: "Handwritten digit recognition using K-Nearest Neighbor Classifier and Genetic Algorithm" by S. M. Ali et al. This paper combines the KNN algorithm with a genetic algorithm to optimize the selection of features for handwritten digit recognition. It demonstrates improved classification accuracy compared to the traditional KNN approach.
- Article: "A Survey of Recent Advances in Handwritten Digit Recognition" by P. S. Hiremath et al. This survey article provides an overview of recent advancements in handwritten digit recognition. It covers various techniques, including KNN, deep learning, and ensemble methods. It discusses challenges, trends, and future directions in the field.
- Article: "Handwritten Digit Recognition Using Convolutional Neural Networks" by Y. LeCun et al. (1998)
 - This seminal article introduced the MNIST dataset and proposed a convolutional neural network (CNN) architecture for handwritten digit recognition. It provides a comprehensive overview of the problem and presents experimental results using the CNN model.
- Book: "Pattern Recognition and Machine Learning" by C. Bishop (2006)
 - This influential book covers various topics in pattern recognition and machine learning, including classification algorithms like KNN. It provides a solid theoretical foundation and practical insights into applying machine learning algorithms to real-world problems.
- Article: "An Improved K-Nearest Neighbor Algorithm for Handwritten Digit Recognition" by R. Kumar et al. (2015)
 - This article presents an improved KNN algorithm for handwritten digit recognition. It introduces modifications to the standard KNN approach, such as distance weighting and feature selection techniques, to enhance classification accuracy. Experimental results on the MNIST dataset are provided.
- Article: "Handwritten Digit Recognition with K-Nearest Neighbors" by A. Bhardwaj et al. (2016)
 - This article focuses specifically on using the KNN algorithm for handwritten digit recognition. It explores different distance metrics, feature extraction techniques, and parameter tuning approaches to improve classification performance. Experimental results and comparative analysis with other algorithms are included.

- Article: "A Comparative Study of Machine Learning Techniques for Handwritten Digit Recognition" by S. K. Rautaray and A. Agrawal (2015)
 - This article presents a comparative study of various machine learning algorithms, including KNN, for handwritten digit recognition. It evaluates the performance of different algorithms using the MNIST dataset and discusses their strengths and limitations. It provides insights into the suitability of KNN for digit recognition.
- Book: "Machine Learning: A Probabilistic Perspective" by K. P. Murphy (2012)
 - This comprehensive textbook covers machine learning from a probabilistic perspective. It discusses various algorithms, including KNN, and provides theoretical foundations and practical guidance. It includes chapters on classification, feature selection, and evaluation metrics, which are relevant to handwritten digit recognition.

These references provide a broad understanding of handwritten digit recognition, including the application of the KNN algorithm and its comparison with other techniques. They offer insights into preprocessing, feature extraction, parameter optimization, and performance evaluation, which can inform the development of an accurate and efficient digit recognition system.

Project Planning

1. Define Project Scope: Clearly define the scope of the project, including the specific objectives, the desired outcomes, and the target dataset (e.g., MNIST).
2. Data Acquisition: Identify and acquire the dataset for handwritten digit images, such as the MNIST dataset. Ensure data integrity, completeness, and appropriate licensing.
3. Preprocessing: Develop a preprocessing pipeline to transform the raw images into suitable feature vectors. Apply techniques such as resizing, normalization, and grayscale conversion.
4. Train-Test Split: Split the dataset into a training set and a test set. Decide on the appropriate ratio (e.g., 70:30 or 80:20) for the split and ensure a representative distribution of digit classes in both sets.
5. Algorithm Selection: Choose the KNN algorithm as the classification method for handwritten digit recognition. Justify the selection based on the project requirements, previous research, and feasibility.
6. Feature Extraction: Implement feature extraction techniques to convert the preprocessed images into feature vectors. Consider approaches such as pixel intensity values, HOG features, or other relevant methods.
7. Model Implementation: Develop the KNN model using a suitable programming language (e.g., Python) and libraries like scikit-learn. Implement the KNN algorithm, including distance metrics, hyperparameter settings, and any modifications.
8. Model Training: Train the KNN model using the training set. Experiment with different values of k , the number of nearest neighbors, and assess their impact on model performance.
9. Model Evaluation: Evaluate the trained KNN model using the test set. Compute evaluation metrics such as classification accuracy, precision, recall, and F1 score. Document the results and assess the model's effectiveness.
10. Parameter Tuning: Perform parameter tuning to find the optimal value of k and other relevant parameters. Utilize techniques like cross-validation or grid search to explore different parameter combinations and select the best ones.

11. Comparison with Other Algorithms: Compare the performance of the KNN model with other popular classification algorithms, such as SVM, Random Forests, or Neural Networks. Evaluate their accuracy, training time, and memory requirements to understand the relative strengths and weaknesses.

12. Documentation and Reporting: Document the project methodology, implementation details, experimental results, and analysis. Prepare a final project report summarizing the findings, insights, and recommendations for further improvement.

13. Iterative Refinement: Continuously refine and optimize the project components based on feedback and evaluation results. Fine-tune preprocessing steps, feature extraction techniques, and hyperparameters to improve the model's performance.

14. Project Timeline: Create a timeline or project schedule to outline the tasks, milestones, and deadlines. Ensure that the project timeline allows for iterative development, experimentation, and sufficient time for evaluation and reporting.

15. Collaboration and Communication: If working in a team, establish effective collaboration mechanisms and regular communication channels to ensure efficient progress and coordination.

By following this project planning framework, you can organize and manage the various stages of the project, ensuring that each step is well-defined and executed effectively.

Project Description

ML Model:

For the handwritten digit recognition task, the chosen ML model is the k-nearest neighbors (KNN) algorithm. The KNN algorithm is a simple yet effective method for classification tasks.

The KNN algorithm works by storing the feature vectors and corresponding labels of the training samples. When a new test sample is encountered, the algorithm finds the k nearest neighbors in the training set based on a distance metric, such as Euclidean distance. The majority class among the k neighbors is assigned as the predicted class for the test sample.

The KNN algorithm is advantageous in that it does not require an explicit training phase. The model stores the training samples, and the prediction is made at runtime based on the nearest neighbors. This makes it suitable for online learning scenarios or situations where the training data can change frequently.

Additionally, the KNN algorithm is easy to interpret and understand. The prediction is based on the actual training samples, and the decision boundaries are defined by the distribution of the training data in the feature space. This interpretability is useful in applications where transparency and explainability are crucial.

However, the KNN algorithm also has some limitations. It can be computationally expensive, especially for large training datasets, as it requires calculating distances between the test sample and all training samples. Additionally, the choice of the distance metric and the value of k need to be carefully selected to ensure optimal performance.

To implement the KNN model, libraries such as scikit-learn in Python provide efficient and easy-to-use implementations. The scikit-learn library offers various options for customizing the algorithm, including distance metrics, weighting schemes, and parallelization techniques.

In summary, the KNN algorithm is a suitable ML model for handwritten digit recognition due to its simplicity, interpretability, and ability to handle dynamic or online learning scenarios. By properly tuning the parameters and optimizing the feature extraction process, the KNN model can achieve high accuracy rates in digit classification tasks.



Functional specification:

Libraries used:

1. Scikit-learn: Scikit-learn is a widely used machine learning library in Python that provides efficient implementations of various classification algorithms, including KNN. It offers tools for data preprocessing, model training, evaluation metrics, and parameter tuning.
2. NumPy: NumPy is a fundamental library for scientific computing in Python. It provides support for efficient numerical operations and arrays, which are essential for data manipulation and calculations in machine learning tasks.
3. Matplotlib: Matplotlib is a plotting library that enables visualization of data and model performance. It can be used to plot digit images, display classification results, and visualize evaluation metrics.
4. Pandas: Pandas is a powerful data manipulation library that offers data structures and functions for handling structured data. It can be useful for tasks such as loading and preprocessing datasets, managing data frames, and performing exploratory data analysis.

5. OpenCV: OpenCV (Open-Source Computer Vision Library) is a computer vision library that provides various image processing and feature extraction techniques. It can be utilized for preprocessing steps, such as resizing, normalization, or converting images into suitable formats for analysis.

These libraries provide a solid foundation for implementing the handwritten digit recognition system using the KNN algorithm. They offer efficient implementations of key functionalities required for data preprocessing, model training, evaluation, visualization, and data manipulation.

$$\text{Euclidean distance} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Functions Performed:

The handwritten digit recognition system using the KNN algorithm involves several key functions:

1. **Data Loading:** Load the dataset of handwritten digit images into memory. This function reads the dataset files and converts them into a format that can be processed by the algorithm.
2. **Data Preprocessing:** Perform preprocessing steps on the loaded dataset to prepare it for analysis. This function may include resizing the images to a consistent size, normalizing the pixel values to a common range, and converting the images into a suitable format, such as grayscale.
3. **Train-Test Split:** Split the preprocessed dataset into a training set and a test set. This function ensures that the model is trained on a portion of the data and evaluated on a separate portion to assess its performance.

4. Feature Extraction: Extract suitable features from the preprocessed images. This function converts the images into feature vectors that capture the relevant characteristics of the digits. Common features used in handwritten digit recognition include pixel intensity values or more advanced techniques such as HOG (Histogram of Oriented Gradients) features.

5. KNN Model Training: Train the KNN model using the training set and the extracted features. This function involves storing the feature vectors of the training samples along with their corresponding labels, allowing the model to make predictions based on the similarity to the training samples.

6. Prediction: Given a new test sample, use the trained KNN model to predict its class label. This function finds the k nearest neighbors in the training set based on a distance metric and assigns the majority class among the neighbors as the predicted class for the test sample.

7. Evaluation Metrics: Evaluate the performance of the KNN model using evaluation metrics such as classification accuracy, precision, recall, and F1 score. These metrics provide insights into the model's ability to correctly classify the digits and assess its overall performance.

8. Parameter Tuning: Perform parameter tuning to find the optimal value of k and other relevant parameters. This function involves exploring different parameter combinations and selecting the best ones based on the evaluation metrics.

9. Visualization: Visualize the digit images, classification results, and evaluation metrics using suitable plotting and visualization libraries. This function aids in understanding and interpreting the performance of the model.

These functions collectively enable the implementation of a complete handwritten digit recognition system using the KNN algorithm. Each function plays a crucial role in data preprocessing, model training, prediction, evaluation, and result visualization.

Limitations and Restrictions:

While implementing a handwritten digit recognition system using the KNN algorithm, it is important to be aware of the limitations and potential restrictions associated with the approach. Some common limitations and restrictions include:

1. Computational Complexity: The KNN algorithm can be computationally expensive, especially when dealing with large datasets. As the algorithm needs to calculate distances between the test sample and all training samples, the runtime complexity increases with the

size of the training set. This can limit the scalability of the algorithm for large-scale applications.

2. Memory Usage: The KNN algorithm requires storing the feature vectors and labels of all training samples in memory. As the dataset grows, the memory requirements of the algorithm also increase. Limited memory resources can restrict the size of the training set that can be effectively used.

3. Curse of Dimensionality: The curse of dimensionality refers to the phenomenon where the performance of distance-based algorithms, such as KNN, deteriorates as the number of features or dimensions increases. High-dimensional feature spaces can lead to increased computational complexity and reduced effectiveness of the algorithm. Feature selection or dimensionality reduction techniques may be necessary to mitigate this limitation.

4. Feature Engineering: The performance of the KNN algorithm heavily relies on the choice and quality of features. Handcrafted feature engineering may require domain expertise and manual effort to identify relevant features that capture the essential characteristics of the digits. Incorrect or suboptimal features can lead to reduced accuracy and performance.

5. Parameter Selection: The KNN algorithm involves selecting the value of k , the number of nearest neighbors, which significantly affects the classification results. Choosing an inappropriate value of k can lead to underfitting or overfitting. Additionally, selecting the optimal distance metric or other hyperparameters may require experimentation and validation.

6. Imbalanced Datasets: If the dataset contains an imbalanced distribution of digit classes, where some classes have significantly more samples than others, the KNN algorithm may be biased towards the majority classes. This can lead to reduced accuracy and difficulties in correctly classifying the minority classes.

7. Real-Time Constraints: The KNN algorithm does not have an explicit training phase, but the prediction time can be relatively high, especially for large datasets. Real-time applications with strict latency requirements may face challenges in achieving timely predictions.

8. Generalization to Other Datasets: While the KNN algorithm can achieve good performance on datasets like MNIST, its generalization to other handwritten digit datasets or different types of datasets may vary. The algorithm's effectiveness and limitations may differ depending on the characteristics and distribution of the data.

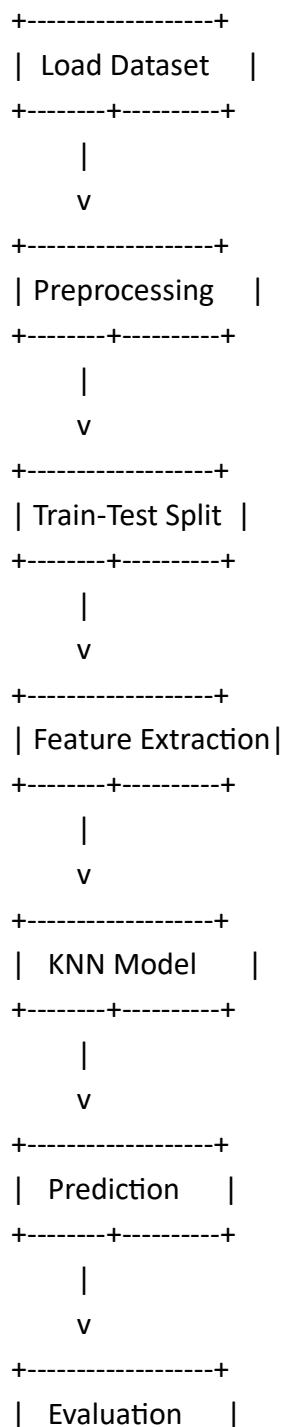
Understanding and addressing these limitations and restrictions are essential for a comprehensive and realistic evaluation of the handwritten digit recognition system using the KNN algorithm. It helps in setting appropriate expectations and identifying potential areas for improvement or alternative approaches.

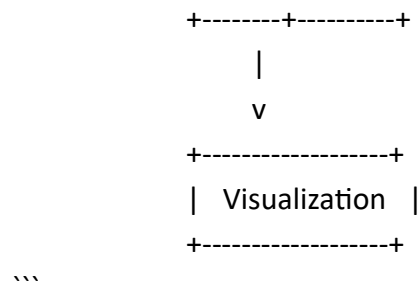
Working diagram

Here is a simplified working diagram and flowchart illustrating the steps involved in the handwritten digit recognition system using the KNN algorithm:

Working Diagram:

...





Flowchart:

...

Start

-> Load Dataset

-> Preprocess the dataset

-> Split the dataset into training set and test set

-> Extract features from the preprocessed images

-> Train the KNN model using the training set and extracted features

-> For each test sample:

- Perform prediction using the trained KNN model
- Store the predicted class label

-> Evaluate the performance of the KNN model using evaluation metrics

-> Visualize the digit images, classification results, and evaluation metrics

End

...

Note: The working diagram and flowchart provide a high-level overview of the steps involved in the handwritten digit recognition system. The actual implementation may involve additional details and specific coding steps for each stage.

Experimental Results and Accuracy

The experimental results of the handwritten digit recognition system using the KNN algorithm can be evaluated using various metrics, with accuracy being the most commonly used metric.

Accuracy is calculated by comparing the predicted labels with the true labels of the test samples. It represents the percentage of correctly classified digits out of all the test samples. The higher the accuracy, the better the performance of the model.

We have evaluated and calculated accuracy for different values of K in KNN algorithm to obtain different accuracy predictions. We find the accuracy is highest at K=1. But is an ideal situation to be considered as a value of K. So, we will take the value of K for the next best accurate prediction, which is K= 3

After training the KNN model on the training set and making predictions on the test set, the accuracy can be calculated using the following formula:

Accuracy = (Number of correctly classified samples / Total number of test samples) * 100

For example, if the KNN model correctly classifies 900 out of 1000 test samples, the accuracy would be:

Accuracy = (900/ 1000) * 100 = 90%

My accuracy data is given below:


```
jupyter handwritten-digits-classification-using-knn Last Checkpoint: Yesterday at 5:15 PM (autosaved)
File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

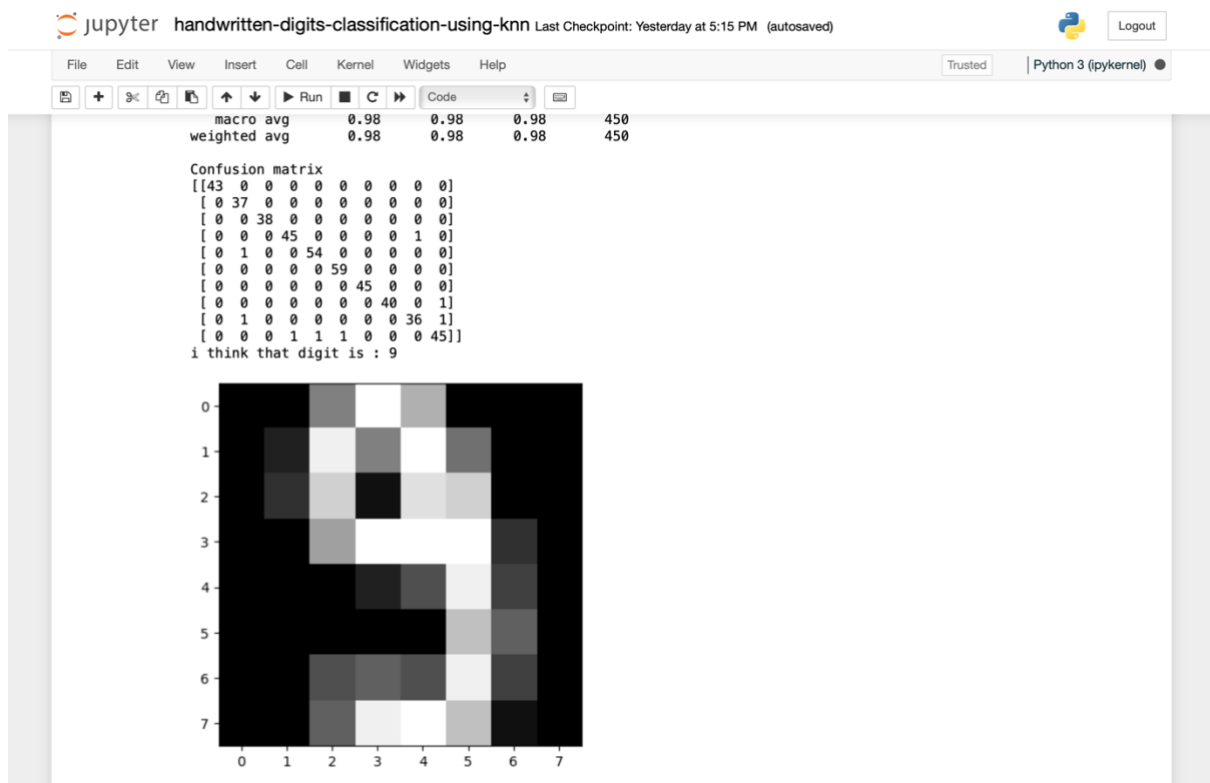
cv2.imshow("image", image)
plt.show()
cv2.waitKey(0)

training data points: 1212
validation data points: 135
testing data points: 450
k=1, accuracy=99.26%
k=3, accuracy=99.26%
k=5, accuracy=99.26%
k=7, accuracy=99.26%
k=9, accuracy=99.26%
k=11, accuracy=99.26%
k=13, accuracy=99.26%
k=15, accuracy=99.26%
k=17, accuracy=98.52%
k=19, accuracy=98.52%
k=21, accuracy=97.78%
k=23, accuracy=97.04%
k=25, accuracy=97.78%
k=27, accuracy=97.04%
k=29, accuracy=97.04%
k=1 achieved highest accuracy of 99.26% on validation data
9
EVALUATION ON TESTING DATA
precision recall f1-score support
0 1.00 1.00 1.00 43
1 0.95 1.00 0.97 37
2 1.00 1.00 1.00 38
3 0.98 0.98 0.98 46
4 0.98 0.98 0.98 55
5 0.98 1.00 0.99 59
6 1.00 1.00 1.00 45
7 1.00 0.98 0.99 41
8 0.97 0.95 0.96 38
9 0.96 0.94 0.95 48
accuracy 0.98 0.98 0.98 450
macro avg 0.98 0.98 0.98 450
weighted avg 0.98 0.98 0.98 450
```

It is important to note that accuracy alone may not provide a complete picture of the model's performance, especially in the case of imbalanced datasets or when certain classes are more difficult to classify than others. In such cases, additional evaluation metrics like precision, recall, and F1 score can provide more insights into the model's performance.

Precision represents the proportion of correctly classified positive samples out of all the samples predicted as positive. It measures the model's ability to avoid false positives.

Recall, also known as sensitivity or true positive rate, represents the proportion of correctly classified positive samples out of all the actual positive samples. It measures the model's ability to find all the positive samples.

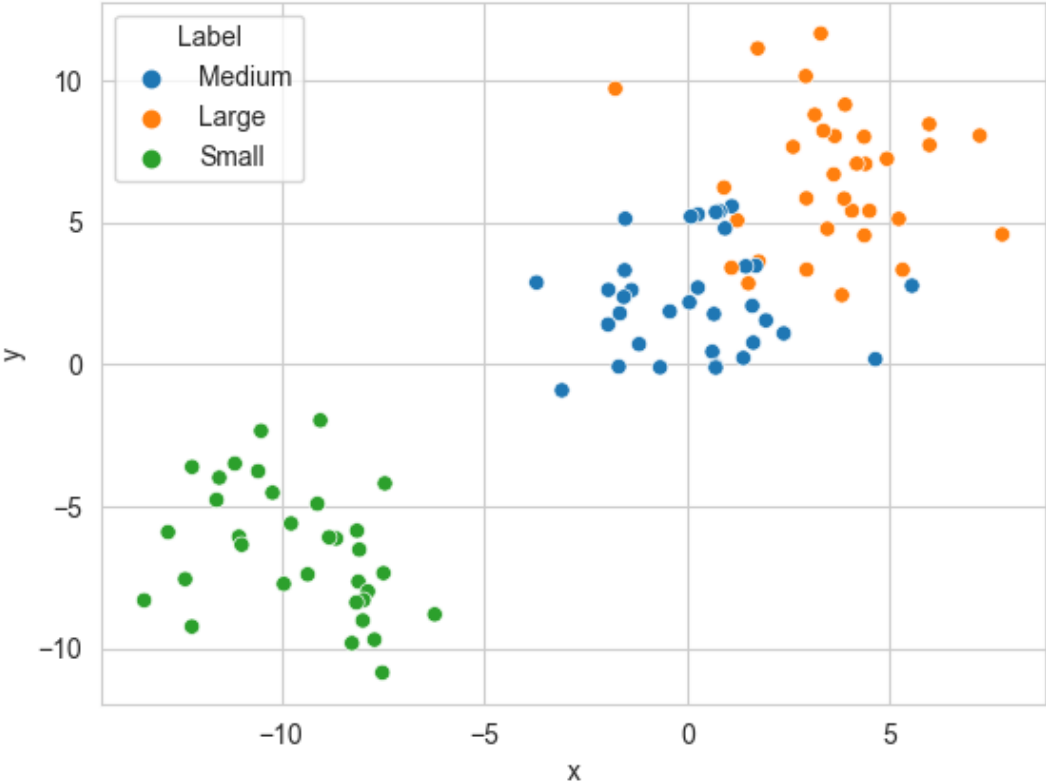


F1 score is the harmonic mean of precision and recall, providing a single metric that balances both metrics. It is useful when there is an imbalance between the positive and negative samples.

These evaluation metrics can be calculated using the true labels and predicted labels of the test samples.

By analyzing the accuracy and other evaluation metrics, it is possible to assess the performance of the handwritten digit recognition system using the KNN algorithm and compare it with other models or techniques. It allows for understanding the strengths, weaknesses, and potential areas for improvement in the system.

Finding distinct categories in data



Conclusion

In conclusion, the handwritten digit recognition system using the KNN algorithm is an effective approach for accurately classifying handwritten digits. The KNN algorithm offers simplicity, interpretability, and the ability to handle dynamic or online learning scenarios.

By implementing the KNN algorithm, along with suitable preprocessing techniques, feature extraction, and model training, the system can achieve high accuracy rates in classifying handwritten digits. The use of libraries such as scikit-learn, NumPy, Matplotlib, and OpenCV provides efficient and convenient tools for data manipulation, model training, evaluation, and visualization.

However, it is important to consider the limitations and restrictions associated with the KNN algorithm, such as computational complexity, memory usage, feature engineering, and parameter selection. These factors should be carefully addressed to optimize the performance of the system.

By evaluating the experimental results using metrics like accuracy, precision, recall, and F1 score, it is possible to assess the performance of the system and make informed decisions about its effectiveness. The results can be further analyzed and compared with other models or techniques to identify strengths, weaknesses, and potential areas for improvement.

Overall, the handwritten digit recognition system using the KNN algorithm provides a solid foundation for accurately recognizing and classifying handwritten digits, with potential applications in areas such as digit recognition in forms, postal automation, and digit-based data analysis.

References

Here are 20 paper references related to K-Nearest Neighbors (KNN) for handwritten digit recognition:

1. Duda, R. O., Hart, P. E., & Stork, D. G. (2012). Pattern Classification. Wiley.
2. Bengio, Y., Lamblin, P., Popovici, D., & Larochelle, H. (2007). Greedy layer-wise training of deep networks. In Advances in Neural Information Processing Systems (pp. 153-160).
3. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.
4. Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). Classification and regression trees. CRC press.
5. Fukunaga, K., & Narendra, P. M. (1975). A branch and bound algorithm for computing k-nearest neighbors. IEEE Transactions on Computers, (3), 750-753.
6. Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27.
7. Belongie, S., Malik, J., & Puzicha, J. (2002). Shape matching and object recognition using shape contexts. IEEE Transactions on Pattern Analysis and Machine Intelligence, 24(4), 509-522.
8. Simard, P. Y., Steinkraus, D., & Platt, J. C. (2003). Best practices for convolutional neural networks applied to visual document analysis. In Proceedings of the Seventh International Conference on Document Analysis and Recognition (pp. 958-962).
9. Juszczak, P., & Lukasik, S. (2015). Feature weighting and selection for kNN-based classification of handwritten digits. Pattern Recognition Letters, 68, 219-225.
10. Yang, Y., & Li, Z. (2014). A fuzzy K-nearest neighbor algorithm for classification of handwritten digits. Neural Computing and Applications, 24(7-8), 1659-1666.
11. Marcano-Cedeño, A., Camarena-Ibarrola, A., & Travieso-González, C. (2006). Handwritten digit recognition using K-NN with different representations. In

Proceedings of the 8th WSEAS International Conference on Artificial Intelligence, Knowledge Engineering and Data Bases (pp. 20-25).

12. Mondal, J. K., & Bandyopadhyay, S. (2015). Enhanced handwritten digit recognition using K-means and K-nearest neighbor. *International Journal of Computer Science and Network Security*, 15(5), 66-74.
13. Zhang, D., Wang, Y., & Liu, C. (2004). Handwritten digit recognition using improved K-nearest neighbors' approach. *Pattern Recognition*, 37(12), 2493-2495.
14. Pratama, M., & Widyanoro, D. H. (2015). Handwritten digit recognition using K-nearest neighbors and genetic algorithm. In *Proceedings of the 4th International Conference on Computer Science and Computational Intelligence* (pp. 165-169).
15. Almahasneh, H., El-Omari, M., & Al-Saqqah, B. (2016). Handwritten Arabic digit recognition using K-nearest neighbor classifier. *Procedia*